Multi-Canonical Dynamic Gaussian Splatting for Accurate Geometry and High-Fidelity Rendering

IEEE Publication Technology, *Staff, IEEE,*

***Abstract*—While 3D Gaussian Splatting (3DGS) has enabled high-quality reconstruction of dynamic scenes, a persistent tension remains between Novel View Synthesis (NVS) fidelity and mesh geometric accuracy. We argue that this arises because deformation modeling, surface alignment, and appearance expressiveness compete for shared representational degrees of freedom. To mitigate this rendering–geometry trade-off, we propose Multi-Canonical Dynamic Gaussian Splatting. Multi-Canonical Deformation Modeling (MCDM) partitions the sequence into localized temporal segments to reduce motion over-smoothing. Surface-Aligned Anisotropic Densification (SAAD) initializes 2D Gaussians with scales and rotations derived from local face geometry for more faithful surface support. Residual-Based Expressiveness Restoration (RBER) restores fine appearance details through sequential 3D and 2D residual refinement on top of a geometry-stabilized base. Experiments on DG-Mesh, D-NeRF, and Nerfies show that our method improves the rendering–geometry balance, yielding consistent gains in rendering quality while preserving or improving mesh accuracy.**

***Index Terms*—Dynamic Scene Reconstruction, Geometry Consistency, Gaussian Splatting, Multi-Canonical.**

I. Introduction

Reconstructing dynamic scenes with both high-fidelity rendering and geometrically consistent mesh surfaces remains a central challenge in dynamic novel view synthesis (NVS). Capturing temporally evolving geometry while preserving fine-scale appearance requires reconciling competing modeling objectives within a single representation. Dynamic NeRF-based methods [1]–[9] represent time-varying scenes through continuous volumetric functions, achieving high-quality renderings but often relying on implicit surface representations and computationally intensive optimization. More recent dynamic Gaussian-based approaches [10]–[14] instead model scenes using deformable primitives with explicit spatial parameterization, enabling direct geometric manipulation and efficient rendering.

However, achieving high-quality rendering and reliable mesh reconstruction simultaneously remains difficult. As illustrated in Fig. 1 (left), rendering-oriented approaches such as HexPlane [4] produce visually sharp Gaussian splatting results but yield fragmented or unstable meshes after surface extraction. In contrast, geometry-focused approaches such as DG-Mesh [15] generate coherent mesh surfaces but noticeably degrade rendering fidelity. Dynamic-2DGS [16] provides moderate performance in both aspects without clearly resolving the tension between them. The quantitative comparison in Fig. 1 (right) further highlights this phenomenon: existing methods distribute along a trade-off between rendering quality (PSNR) and mesh accuracy (PSNR_m).

We argue that this *trade-off* arises from how deformation, geometry, and appearance are jointly optimized. In dynamic Gaussian reconstruction, deformation modeling, surface alignment, and appearance representation are typically coupled within a shared parameterization. A single canonical Gaussian set is often required to accommodate substantial temporal variation, while geometry supervision constrains primitive positions and covariances to align with reconstructed surfaces. As a result, deformation capacity, surface consistency, and appearance expressiveness compete for the same representational degrees of freedom. Increasing geometric constraints can suppress fine-scale appearance variation, whereas relaxing them may compromise mesh stability.

Instead of addressing these effects through isolated modifications, we revisit the reconstruction pipeline from a structural perspective. Our analysis identifies three interacting factors underlying the trade-off: (1) excessive temporal deformation span handled by a shared canonical representation, (2) geometry-driven bias introduced during primitive densification, and (3) attenuated representational expressiveness under strong surface supervision. Based on this analysis, we introduce a structural reformulation of dynamic Gaussian reconstruction that alleviates the competition among deformation modeling, geometry supervision, and appearance expressiveness within a shared optimization framework. Specifically, our contributions are threefold. We identify excessive temporal deformation span within a shared canonical representation as a key source of over-smoothing. First, we address this by introducing **Multi-Canonical Deformation Modeling (MCDM)**, which redistributes temporal modeling capacity across multiple canonical Gaussian sets to stabilize dynamic reconstruction. Second, we analyze the bias introduced by conventional densification under surface constraints and propose **Surface-Aligned Anisotropic Densification (SAAD)**, which aligns primitive covariance with local surface statistics to preserve structural detail under differentiable mesh supervision. Third, we enhance reconstruction fidelity under geometry-supervised optimization by introducing **Residual-Based Expressiveness Restoration (RBER)**, which restores fine-scale detail via residual refinement.

Together, these components reorganize dynamic Gaussian reconstruction into a staged modeling process that reduces representational competition and enables coordinated improvements in mesh accuracy and rendering fidelity. Extensive experiments on dynamic reconstruction benchmarks demonstrate consistent improvements in both mesh accuracy and

This paper was produced by the IEEE Publication Technology Group. They are in Piscataway, NJ.

Manuscript received April 19, 2021; revised August 16, 2021.

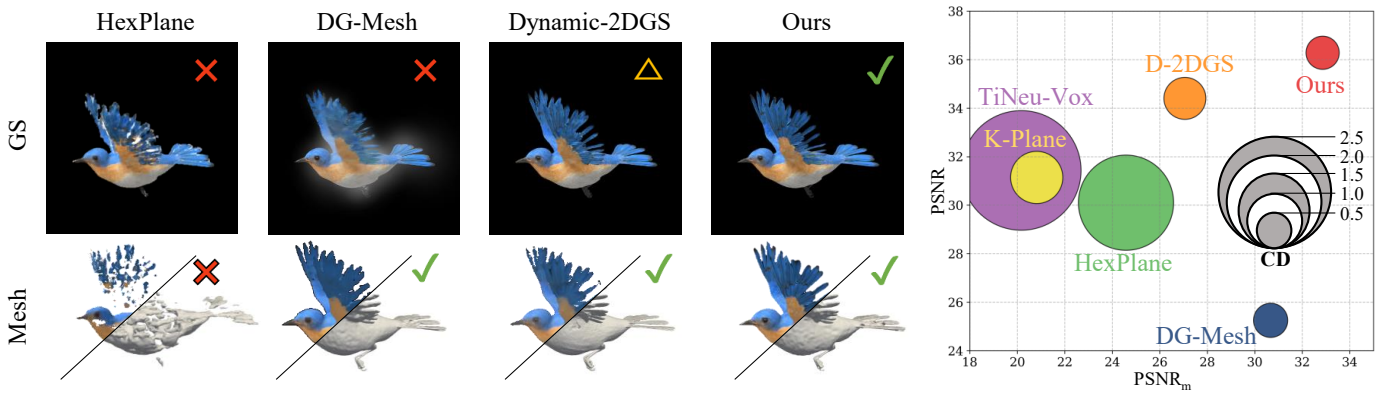


Fig. 1. **Results of mesh and Gaussian Splatting (GS) rendering comparison.** We illustrate a commonly observed trade-off between Novel View Synthesis (NVS) rendering quality and mesh geometric accuracy. ✓ indicates strong performance, × denotes a clear weakness, and △ suggests moderate results. While rendering-focused methods such as HexPlane [4] exhibit geometric flaws, geometry-focused methods such as DG-Mesh [15] and Dynamic-2DGS [16] show limited or moderate rendering quality. Our method achieves strong performance in both aspects, substantially improving the rendering–geometry balance.

rendering quality compared to representative NeRF [17]-based and Gaussian splatting [18]-based baselines.

II. Related Works

A. Novel view synthesis in Dynamic Scene

The task of synthesizing novel views for dynamic scenes has advanced rapidly with the development of neural rendering techniques. Early works extend neural radiance fields (NeRF) [17] to dynamic settings [1]–[9] by introducing temporal deformation models that map time-varying observations into a shared canonical space, where static NeRFs are learned [1], [2], [5], [19], [20]. They follow this canonical-space deformation paradigm to capture non-rigid motion while preserving view-consistent appearance. Another line of research improves efficiency by decomposing the 4D space-time representation into lower-dimensional structures [3], [4], [21], [22]. Methods such as K-Planes [3] and HexPlanes [4] factorize the dynamic radiance field into planar or grid-based components, enabling faster training and rendering while maintaining competitive reconstruction quality. More recently, the introduction of 3D Gaussian splatting (3DGS) [18] has led to a paradigm shift towards explicit representations. In dynamic settings, these approaches model temporal evolution by predicting per-frame transformations of Gaussian primitives, including translation, rotation, and scale [10]–[14]. Methods such as 4DGS [11] and Deformable 3DGS [10] employ deformation networks to map canonical Gaussians to time-specific configurations, while control-oriented frameworks like SC-GS [23] improve editability and scene manipulation. Although these approaches achieve high-quality NVS, they primarily focus on rendering fidelity. As a result, the geometric structure of the reconstructed scene is often weakly constrained, making accurate mesh extraction challenging.

B. Mesh Reconstruction in Dynamic Scene

While novel view synthesis is the main objective of many dynamic reconstruction methods, extracting accurate and temporally consistent meshes remains essential for compatibility with conventional graphics pipelines and downstream applications. Template-based methods [24]–[29], deform a predefined base mesh to represent a target geometry. However, they often rely on category-specific priors and struggle when the scene undergoes large non-rigid deformations or topology changes. Another line of work reconstructs surfaces from implicit neural representations. In NeRF-based frameworks [30]–[33], surface geometry is typically recovered by learning a signed distance function (SDF) and extracting the zero-level set [33] using algorithms such as marching cubes [34]. Although this approach enables flexible surface modeling, the geometry is derived indirectly from volumetric representations and often requires additional regularization or post-processing to ensure stable surfaces. Recent studies attempt to improve geometric reconstruction by introducing surface-aligned Gaussian representations [35]–[40]. For example, DG-Mesh [15] constrains Gaussian primitives to remain close to mesh surfaces through Gaussian–mesh anchoring, encouraging geometric smoothness. MaGS [41] allows Gaussians to move near a guiding mesh to maintain structural organization, while Dynamic 2DGS [16] employs surfel-based planar primitives [36] that explicitly promote planar surface alignment via truncated SDF (TSDF) [42]. While these methods improve mesh reconstruction accuracy, enforcing strong geometric constraints inevitably restricts the flexibility of Gaussian parameters used for appearance modeling. Consequently, many geometry-oriented approaches achieve reliable meshes but suffer from degraded rendering fidelity, leading to a trade-off between geometric accuracy and rendering quality.

III. Preliminaries

Throughout the paper, $\mathbf{x} \in \mathbb{R}^2$ denotes an image-plane pixel coordinate, $\mathbf{u} = (u, v) \in \mathbb{R}^2$ denotes a local coordinate on a Gaussian plane, and $\mathbf{p} \in \mathbb{R}^3$ denotes a 3D point in world coordinates. We use $\mathcal{R}(\cdot)$ to denote 2D Gaussian rendering and $\mathcal{M}(\cdot)$ to denote differentiable mesh extraction.

A. 2D Gaussian Splatting

Unlike standard 3D Gaussian Splatting, whose volumetric ellipsoids distribute density along all three axes and often yield

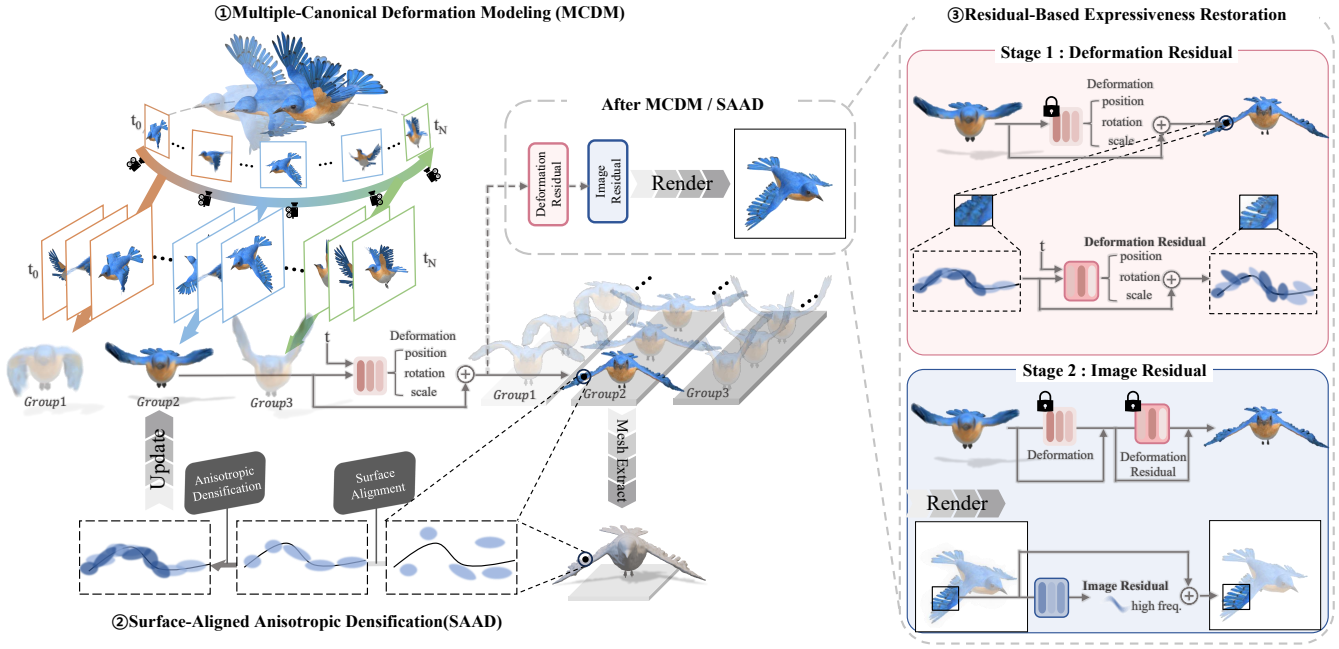


Fig. 2. **Overview of the proposed framework.** Our pipeline seamlessly integrates three core modules. **MCDM** utilizes multiple canonical Gaussian sets for local temporal modeling, which are deformed via a Deform network f_D. Based on the extracted Deformed Mesh from Deformed Gaussian, **SAAD** densifies new anisotropic Gaussians by aligning their covariance with local surface geometry. Furthermore, **RBER** leverages Deformation Residual network f_{DR} for parameter residual $(\Delta\mathbf{p}, \Delta\mathbf{s}, \Delta\mathbf{r})$ and Local Texture estimator f_{lte} to recover high-frequency appearance details.

view-inconsistent depth, 2D Gaussian Splatting (2DGS) [36] represents each primitive as an oriented planar disk aligned with a local surface tangent plane. We write the i-th Gaussian as

$$g_i = (\mathbf{p}_i, \mathbf{r}_i, \mathbf{s}_i, \alpha_i, \mathbf{c}_i), \tag{1}$$

where $\mathbf{p}_i \in \mathbb{R}^3$ is the Gaussian center, $\mathbf{r}_i$ is a quaternion parameterizing the local orientation, $\mathbf{s}_i = (s_{i,u}, s_{i,v})$ is the in-plane scale, α_i is opacity, and $\mathbf{c}_i$ denotes appearance parameters. The quaternion $\mathbf{r}_i$ defines an orthonormal basis $(\mathbf{t}_{i,u}, \mathbf{t}_{i,v}, \mathbf{t}_{i,w})$, where $\mathbf{t}_{i,w} = \mathbf{t}_{i,u} \times \mathbf{t}_{i,v}$. Following 2DGS, the i-th Gaussian is defined on a local tangent plane in world space as

$$P_i(u,v) = \mathbf{p}_i + s_{i,u}\mathbf{t}_{i,u}u + s_{i,v}\mathbf{t}_{i,v}v = \mathbf{H}_i(u,v,1,1)^\top, \tag{2}$$

with

$$\mathbf{H}_i = \begin{bmatrix} s_{i,u}\mathbf{t}_{i,u} & s_{i,v}\mathbf{t}_{i,v} & \mathbf{0} & \mathbf{p}_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{3}$$

The corresponding Gaussian weight on the local plane is

$$G_i(\mathbf{u}) = \exp\left(-\frac{1}{2}\|\mathbf{u}\|_2^2\right). \tag{4}$$

For a pixel $\mathbf{x} = (x_1, x_2)$, let $\mathbf{u}_i(\mathbf{x})$ denote the local 2D coordinate of the ray–splat intersection in the tangent plane of the i-th Gaussian, rather than an affine screen-space projection. More precisely, following 2DGS, if $\mathbf{W}$ denotes the world-to-screen transform, then the ray passing through pixel $\mathbf{x}$ and intersecting the i-th Gaussian at depth z satisfies $(x_1 z, x_2 z, z, z)^\top = \mathbf{W}P_i(u,v) = \mathbf{W}\mathbf{H}_i(u,v,1,1)^\top$. Equivalently, the pixel ray can be represented as the intersection of two homogeneous planes $\mathbf{h}_x = (-1,0,0,x_1)^\top$ and $\mathbf{h}_y = (0,-1,0,x_2)^\top$; after transforming them into the local coordinates of the i-th Gaussian as $\mathbf{h}_{u,i} = (\mathbf{W}\mathbf{H}_i)^\top\mathbf{h}_x$ and $\mathbf{h}_{v,i} = (\mathbf{W}\mathbf{H}_i)^\top\mathbf{h}_y$, $\mathbf{u}_i(\mathbf{x}) = (u_i, v_i)$ is obtained by solving $\mathbf{h}_{u,i}\cdot(u,v,1,1)^\top = \mathbf{h}_{v,i}\cdot(u,v,1,1)^\top = 0$. Thus, $G_i(\mathbf{u}_i(\mathbf{x}))$ evaluates the Gaussian weight at the perspective-correct ray–splat intersection in the local tangent plane. After sorting the intersected Gaussians $\mathcal{N}(\mathbf{x})$ by depth, 2DGS renders the pixel color as

$$\mathbf{C}(\mathbf{x}) = \sum_{i\in\mathcal{N}(\mathbf{x})} T_i(\mathbf{x})\,\alpha_i\,G_i\big(\mathbf{u}_i(\mathbf{x})\big)\,\mathbf{c}_i, \tag{5}$$

with accumulated transmittance

$$T_i(\mathbf{x}) = \prod_{j=1}^{i-1}\left(1 - \alpha_j G_j\big(\mathbf{u}_j(\mathbf{x})\big)\right). \tag{6}$$

This surfel-like parameterization makes 2DGS naturally compatible with surface-aware regularization and mesh extraction.

B. Mesh Extraction

A common way to extract a mesh from Gaussian splatting is to fuse multi-view depth maps with TSDF [42] and then apply Marching Cubes. In a post-hoc TSDF pipeline, the mesh extraction step itself is non-differentiable, so losses defined on the extracted mesh cannot be backpropagated through the meshing process. Following recent differentiable surface reconstruction pipelines, we instead convert a time-specific Gaussian set $\mathcal{G}_t$ into a mesh through DPSR [43] and differentiable Marching Cubes (DMC) [44]:

$$M_t = \mathcal{M}(\mathcal{G}_t) = \mathrm{DMC}\big(\mathrm{DPSR}(\mathcal{G}_t)\big). \tag{7}$$

This allows losses defined on the extracted surface to backpropagate directly to the Gaussian parameters, so geometry

supervision actively shapes the representation instead of being imposed only after rendering.

IV. METHOD

Given a monocular dynamic sequence, our goal is to reconstruct a time-varying 2D Gaussian representation that supports accurate Gaussian rendering together with geometrically consistent mesh extraction. Following the analysis in Sec. I, we view the rendering–geometry tension in dynamic Gaussian reconstruction as arising from three coupled factors within a shared parameterization: excessive temporal deformation span, geometry-driven densification bias under mesh supervision, and attenuated appearance expressiveness after strong surface regularization. As summarized in Fig. 2, we therefore reorganize the reconstruction pipeline into a staged process that progressively decouples these burdens. Multi-Canonical Deformation Modeling (MCDM) localizes temporal deformation into multiple canonical spaces. Surface-Aligned Anisotropic Densification (SAAD) corrects the support mismatch introduced by isotropic densification by aligning newly added primitives with local surface statistics. Residual-Based Expressiveness Restoration (RBER) then restores only the residual appearance variation that remains after geometry stabilization through sequential 3D and 2D residual refinement. This design preserves the geometric benefits of 2DGS and differentiable mesh supervision while improving rendering fidelity. In this view, MCDM, SAAD, and RBER should not be interpreted as isolated components, but as sequential mechanisms that decouple temporal deformation, geometry-support formation, and residual appearance modeling within a shared dynamic Gaussian representation.

A. Multi-Canonical Deformation Modeling (MCDM)

MCDM addresses the first factor identified in Sec. I, namely the excessive temporal deformation span imposed on a shared canonical representation. Standard dynamic Gaussian methods maintain a single canonical Gaussian set and require one deformation network to explain the entire temporal motion range. As the sequence becomes longer and the motion becomes more diverse, local temporal patterns compete within the same canonical space, which tends to over-smooth time-specific structures. To redistribute this temporal modeling burden, we first learn a sequence-level global canonical prior and then clone it into multiple localized canonical Gaussian sets, as illustrated in Fig. 3.

Global warm-up. We first optimize a global canonical set

$$\mathcal{G}_{\text{glo}} = \{g_n^{\text{glo}}\}_{n=1}^{N} \tag{8}$$

together with a deformation network f_D using all training frames. Let $\tau_t = (t-1)/(T-1) \in [0,1]$ denote the normalized time of frame t. For each global canonical Gaussian g_n^{glo}, the network predicts deformation offsets from its canonical center and normalized time:

$$\left(\Delta\mathbf{p}_{n,t}^{\text{def}}, \Delta\mathbf{r}_{n,t}^{\text{def}}, \Delta\mathbf{s}_{n,t}^{\text{def}}\right) = f_D\left(\gamma(\mathbf{p}_n^{\text{glo}}), \gamma(\tau_t)\right), \tag{9}$$

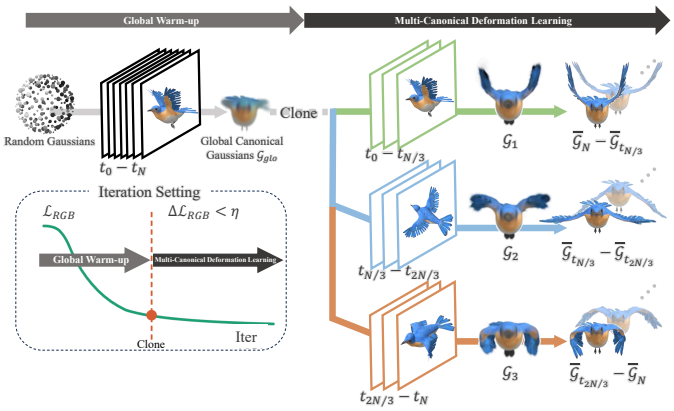


Fig. 3. **Overview of MCDM.** MCDM localizes temporal deformation burden. (a) Global warm-up learns a sequence-level canonical prior $\mathcal{G}_{\text{glo}}$. (b) $\mathcal{G}_{\text{glo}}$ is cloned into multiple canonical Gaussian sets $\mathcal{G}_k$, each assigned to a localized temporal segment. (c) Only the active group is deformed at time t, so each canonical space only needs to explain a reduced motion range.

where $\gamma(\cdot)$ denotes positional encoding. The corresponding deformed parameters are

$$\bar{\mathbf{p}}_{n,t}^{\text{glo}} = \mathbf{p}_n^{\text{glo}} + \Delta\mathbf{p}_{n,t}^{\text{def}}, \tag{10}$$

$$\bar{\mathbf{r}}_{n,t}^{\text{glo}} = \mathbf{r}_n^{\text{glo}} + \Delta\mathbf{r}_{n,t}^{\text{def}}, \tag{11}$$

$$\bar{\mathbf{s}}_{n,t}^{\text{glo}} = \mathbf{s}_n^{\text{glo}} + \Delta\mathbf{s}_{n,t}^{\text{def}}. \tag{12}$$

This yields the warm-up deformation

$$\bar{\mathcal{G}}_t^{\text{glo}} = \left\{\bar{g}_{n,t}^{\text{glo}}\right\}_{n=1}^{N}, \tag{13}$$

which serves as a coarse yet globally consistent sequence-level geometric prior.

Multi-canonical deformation learning. After warm-up, we clone $\mathcal{G}_{\text{glo}}$ into K canonical Gaussian sets

$$\{\mathcal{G}_k\}_{k=1}^{K}. \tag{14}$$

The canonical Gaussian set of group k is denoted by

$$\mathcal{G}_k = \{g_{k,n}\}_{n=1}^{N}. \tag{15}$$

We assign each frame to a group by

$$k(t) = \min\left(\lfloor \tau_t K \rfloor + 1, K\right). \tag{16}$$

For frame t, we activate only the corresponding set $\mathcal{G}_{k(t)}$, while sharing the deformation network f_D across all groups. The common initialization from $\mathcal{G}_{\text{glo}}$ provides a smooth initialization across neighboring groups and empirically reduces abrupt discrepancies at group boundaries, while the group-specific canonical sets prevent a single canonical space from absorbing the full sequence-wide motion.

For each Gaussian $g_{k(t),n} \in \mathcal{G}_{k(t)}$, the shared deformation network predicts

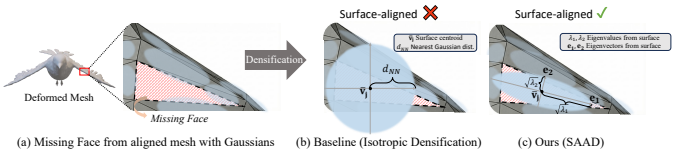
$$\left(\Delta\mathbf{p}_{n,t}^{\text{def}}, \Delta\mathbf{r}_{n,t}^{\text{def}}, \Delta\mathbf{s}_{n,t}^{\text{def}}\right) = f_D\left(\gamma(\mathbf{p}_{k(t),n}), \gamma(\tau_t)\right). \tag{17}$$

The deformed parameters at time t are then

$$\bar{\mathbf{p}}_{n,t} = \mathbf{p}_{k(t),n} + \Delta\mathbf{p}_{n,t}^{\text{def}}, \tag{18}$$

$$\bar{\mathbf{r}}_{n,t} = \mathbf{r}_{k(t),n} + \Delta\mathbf{r}_{n,t}^{\text{def}}, \tag{19}$$

$$\bar{\mathbf{s}}_{n,t} = \mathbf{s}_{k(t),n} + \Delta\mathbf{s}_{n,t}^{\text{def}}. \tag{20}$$



(a) Missing Face from aligned mesh with Gaussians (b) Baseline (Isotropic Densification) (c) Ours (SAAD)

Fig. 4. **Overview of SAAD.** (a) Unrepresented mesh faces lacking corresponding 2D Gaussians are identified (Missing Face). (b) The baseline inserts an isotropic Gaussian at the face centroid $\bar{\mathbf{v}}_j$ based only on the nearest-neighbor distance d_{NN}, which improves coverage but ignores local surface anisotropy. (c) SAAD initializes a surface-aligned anisotropic Gaussian whose tangent axes $(\mathbf{e}_1, \mathbf{e}_2)$ and scales (s_u, s_v) are derived from the eigendecomposition of the face vertex covariance.

We denote the resulting deformed Gaussian set by

$$\bar{\mathcal{G}}_t = \{\bar{g}_{n,t}\}_{n=1}^{N}. \tag{21}$$

Because each canonical set only covers a restricted temporal segment, the shared deformation network no longer needs to explain the entire sequence within one canonical space. In this way, MCDM localizes temporal deformation burden while retaining a common sequence-level prior. The deformed set $\bar{\mathcal{G}}_t$ is then forwarded to the mesh extraction and densification stage.

B. Surface-Aligned Anisotropic Densification (SAAD)

SAAD addresses the second factor in Sec. I, namely the geometry-driven bias introduced during primitive densification under surface supervision. The deformed set $\bar{\mathcal{G}}_t$ is geometrically regularized through differentiable mesh supervision. However, when a mesh face lacks sufficient Gaussian support, conventional densification typically inserts a new primitive at the face centroid with an isotropic scale, ignoring the anisotropic structure of the local surface patch. Such a primitive increases coverage, but its support is poorly matched to the face geometry, which tends to blur local structure, as illustrated in Fig. 4. We instead initialize each densified Gaussian directly from local face statistics so that densification improves both coverage and surface alignment.

We first extract a deformed mesh

$$M_t = \mathcal{M}(\bar{\mathcal{G}}_t). \tag{22}$$

Let $f_j = \{\mathbf{v}_{j,1}, \mathbf{v}_{j,2}, \mathbf{v}_{j,3}\}$ denote a mesh face, and let its centroid be

$$\bar{\mathbf{v}}_j = \frac{1}{3}\sum_{i=1}^{3} \mathbf{v}_{j,i}. \tag{23}$$

We regard f_j as insufficiently supported if its nearest deformed Gaussian center lies farther than a threshold ρ, i.e.,

$$d_j^{\mathrm{NN}} = \min_n \|\bar{\mathbf{p}}_{n,t} - \bar{\mathbf{v}}_j\|_2 > \rho. \tag{24}$$

For each such face, we initialize a new Gaussian from local face statistics. We compute the covariance of its three vertices as

$$\mathbf{C}_j = \frac{1}{3}\sum_{i=1}^{3}(\mathbf{v}_{j,i} - \bar{\mathbf{v}}_j)(\mathbf{v}_{j,i} - \bar{\mathbf{v}}_j)^\top. \tag{25}$$

Its eigendecomposition is

$$\mathbf{C}_j = \mathbf{E}_j \mathbf{\Lambda}_j \mathbf{E}_j^\top, \tag{26}$$

where

$$\mathbf{E}_j = [\mathbf{e}_{j,1}, \mathbf{e}_{j,2}, \mathbf{e}_{j,3}] \tag{27}$$

contains the principal directions and

$$\mathbf{\Lambda}_j = \mathrm{diag}(\lambda_{j,1}, \lambda_{j,2}, \lambda_{j,3}), \qquad \lambda_{j,1} \geq \lambda_{j,2} \geq \lambda_{j,3}, \tag{28}$$

contains the corresponding variances.

Because 2DGS represents an oriented planar primitive, only the tangent-plane statistics should determine the initial support of a new Gaussian. We therefore use the first two principal directions to define the local tangent frame and treat $\mathbf{e}_{j,3}$ as the local normal. To resolve the sign ambiguity of the eigendecomposition, we orient $\mathbf{e}_{j,3}$ to agree with the face normal and, if necessary, flip one tangent direction so that $[\mathbf{e}_{j,1}, \mathbf{e}_{j,2}, \mathbf{e}_{j,3}]$ forms a right-handed frame. The center and scale of the new Gaussian are initialized as

$$\mathbf{p}_j^{\mathrm{new}} = \bar{\mathbf{v}}_j, \qquad \mathbf{s}_j^{\mathrm{new}} = \left(\sqrt{\lambda_{j,1}}, \sqrt{\lambda_{j,2}}\right). \tag{29}$$

The principal frame $[\mathbf{e}_{j,1}, \mathbf{e}_{j,2}, \mathbf{e}_{j,3}]$ is then converted to the rotation parameter $\mathbf{r}_j^{\mathrm{new}}$ used by 2DGS. The newly initialized Gaussian is

$$g_j^{\mathrm{new}} = \left(\mathbf{p}_j^{\mathrm{new}}, \mathbf{r}_j^{\mathrm{new}}, \mathbf{s}_j^{\mathrm{new}}, \alpha_j^{\mathrm{new}}, \mathbf{c}_j^{\mathrm{new}}\right). \tag{30}$$

Collecting all such primitives gives the densified deformed-space set $\mathcal{G}_t^{\mathrm{new}}$, and the current-frame augmented set becomes

$$\bar{\mathcal{G}}_t^{+} = \bar{\mathcal{G}}_t \cup \mathcal{G}_t^{\mathrm{new}}. \tag{31}$$

To make these updates persistent rather than frame-local, each newly added deformed-space Gaussian is projected back to the active canonical set $\mathcal{G}_{k(t)}$ through a backward deformation module, following the cycle-consistent update used in DG-Mesh [15]; the updated canonical Gaussians are then deformed forward again in subsequent iterations. For notational simplicity, we continue to use N to denote the current number of Gaussians in the active set after these cyclic updates. A mesh is then re-extracted from $\bar{\mathcal{G}}_t^{+}$, so densification becomes surface-statistics-aware rather than purely coverage-seeking, iteratively improving both surface coverage and geometric accuracy.

To supervise the resulting mesh, we follow DG-Mesh [15] and optimize

$$\mathcal{L}_{\mathrm{geo}} = \lambda_{\mathrm{anchor}}\mathcal{L}_{\mathrm{anchor}} + \lambda_{\mathrm{mask}}\mathcal{L}_{\mathrm{mask}} + \lambda_{\mathrm{lap}}\mathcal{L}_{\mathrm{lap}}, \tag{32}$$

where $\mathcal{L}_{\mathrm{anchor}}$ minimizes the distance between the densified Gaussians and the extracted mesh surface, $\mathcal{L}_{\mathrm{mask}}$ applies an $\mathcal{L}_1$ silhouette loss to the foreground mask, and $\mathcal{L}_{\mathrm{lap}}$ enforces locally coherent surface geometry via Laplacian smoothing.

C. Residual-Based Expressiveness Restoration (RBER)

RBER addresses the third factor in Sec. I, namely the reduced appearance expressiveness that remains after strong surface supervision. After SAAD, the reconstruction becomes geometry-dominant: mesh supervision and the planar nature of 2DGS constrain Gaussian centers and orientations to remain close to the extracted surface. While this improves geometric accuracy, it reduces the residual degrees of freedom available for fine appearance details. Rather than weakening the geometry supervision itself, RBER restores only the missing

TABLE I

QUANTITATIVE COMPARISON ON THE DG-MESH DATASET. WE EVALUATE MESH GEOMETRY USING CHAMFER DISTANCE (CD) AND EARTH MOVER'S DISTANCE (EMD), WHERE LOWER IS BETTER. WE ALSO ASSESS RENDERING QUALITY VIA PSNR FOR THE DIRECT GAUSSIAN SPLATTING (GS) OUTPUT, AND PSNR_m FOR THE FINAL RENDERED MESH, WHERE HIGHER IS BETTER. FOR EACH METRIC, THE BEST , AND SECOND-BEST RESULTS ARE HIGHLIGHTED ACCORDINGLY.

Method	*Duck*				*Horse*				*Bird*			
	CD↓	EMD↓	PSNR↑	PSNR_m↑	CD↓	EMD↓	PSNR↑	PSNR_m↑	CD↓	EMD↓	PSNR↑	PSNR_m↑
D-NeRF	0.934	0.073	29.79	23.02	1.685	0.280	25.47	17.38	1.532	0.163	23.85	19.57
K-Plane	1.085	0.055	33.36	20.37	1.480	0.239	28.11	21.63	0.742	0.131	23.72	19.56
HexPlane	2.161	0.090	32.11	27.95	1.750	0.199	26.78	22.40	4.158	0.178	22.19	20.60
TiNeuVox-B	0.969	0.059	34.33	22.07	1.918	0.246	28.16	18.16	8.264	0.215	25.55	19.84
Dynamic-2DGS	1.040	0.092	38.95	27.42	0.391	0.177	31.92	24.46	0.328	0.110	30.19	27.68
DG-Mesh	0.782	0.047	28.12	32.76	0.297	0.164	23.44	30.87	0.510	0.125	24.36	28.09
Ours	0.785	0.047	38.93	34.91	0.278	0.169	36.23	33.18	0.423	0.108	31.65	29.43
Method	*Beagle*				*Torus2sphere*				*Girlwalk*			
	CD↓	EMD↓	PSNR↑	PSNR_m↑	CD↓	EMD↓	PSNR↑	PSNR_m↑	CD↓	EMD↓	PSNR↑	PSNR_m↑
D-NeRF	1.001	0.149	34.47	24.45	1.760	0.250	24.23	13.56	0.601	0.190	28.63	21.15
K-Plane	0.810	0.122	38.33	24.61	1.793	0.161	31.22	15.71	0.495	0.173	32.12	23.01
HexPlane	0.870	0.115	38.03	29.97	2.190	0.190	29.71	22.35	0.597	0.155	31.77	24.21
TiNeuVox-B	0.874	0.129	38.97	25.77	2.115	0.203	28.76	14.99	0.568	0.184	32.81	20.21
Dynamic-2DGS	0.544	0.122	40.98	25.24	2.479	0.164	30.17	27.28	0.324	0.129	34.43	30.28
DG-Mesh	0.623	0.114	29.17	33.57	1.572	0.177	20.53	25.70	0.398	0.151	25.85	33.03
Ours	0.607	0.120	38.71	34.60	1.612	0.169	32.24	30.18	0.330	0.142	39.93	34.70

TABLE II

QUANTITATIVE COMPARISON OF RENDERING QUALITY ON THE D-NERF DATASET. SINCE THIS DATASET DOES NOT PROVIDE GROUND-TRUTH MESHES, WE EVALUATE VISUAL FIDELITY. THE DIRECT GAUSSIAN RENDERING QUALITY IS ASSESSED USING PSNR, SSIM (HIGHER IS BETTER), AND LPIPS (LOWER IS BETTER). ADDITIONALLY, THE RENDERING QUALITY OF THE FINAL EXTRACTED MESH IS EVALUATED USING PSNR_m (HIGHER IS BETTER).

Method	*Mutant*				*Standup*				*Trex*				*Hellwarrior*			
	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑
D-NeRF	31.29	0.906	0.077	21.07	32.79	0.925	0.069	23.30	31.75	0.908	0.085	22.59	25.02	0.877	0.129	18.91
K-Plane	32.50	0.923	0.064	23.23	33.10	0.946	0.048	25.78	30.43	0.921	0.075	23.09	24.58	0.881	0.123	18.07
HexPlane	33.67	0.953	0.045	26.81	34.40	0.965	0.035	27.93	30.67	0.953	0.046	26.63	24.55	0.917	0.094	21.25
TiNeuVox-B	30.87	0.925	0.064	22.97	34.61	0.941	0.051	24.26	31.25	0.927	0.070	24.22	27.10	0.883	0.118	18.66
Dynamic-2DGS	37.83	0.995	0.005	28.12	39.07	0.996	0.003	29.51	25.47	0.956	0.048	28.68	32.14	0.985	0.016	25.50
DG-Mesh	31.44	0.969	0.038	30.40	32.31	0.979	0.036	30.21	29.10	0.967	0.051	28.95	29.34	0.961	0.052	25.46
Ours	39.14	0.995	0.006	31.27	40.90	0.996	0.004	34.20	32.92	0.982	0.018	29.01	29.59	0.972	0.034	27.42
Method	*Lego*				*Bouncingballs*				*Jumpingjacks*				*Hook*			
	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑	PSNR↑	SSIM↑	LPIPS↓	PSNR_m↑
D-NeRF	23.82	0.818	0.137	20.38	38.18	0.899	0.157	23.40	33.73	0.914	0.103	22.26	30.42	0.889	0.108	20.30
K-Plane	25.49	0.830	0.127	19.52	40.05	0.935	0.109	23.31	31.11	0.937	0.068	25.24	28.12	0.900	0.094	22.50
HexPlane	25.10	0.904	0.072	22.87	39.86	0.957	0.069	25.39	31.31	0.954	0.052	27.08	28.63	0.929	0.070	24.51
TiNeuVox-B	24.65	0.843	0.126	21.93	40.23	0.947	0.101	24.82	33.49	0.932	0.075	23.62	28.63	0.908	0.085	21.43
Dynamic-2DGS	25.79	0.953	0.040	23.29	41.42	0.996	0.019	27.78	38.30	0.994	0.066	29.29	35.54	0.991	0.009	27.80
DG-Mesh	22.46	0.923	0.064	21.29	31.66	0.976	0.035	29.15	27.54	0.970	0.115	31.77	29.34	0.961	0.052	27.88
Ours	25.28	0.940	0.041	23.56	40.46	0.992	0.019	30.95	37.63	0.991	0.014	33.79	34.19	0.984	0.017	29.15

appearance components through residual channels built on top of the geometry-stabilized base. Specifically, RBER consists of two sequential stages: a 3D Gaussian residual stage followed by a 2D image residual stage.

Stage 1: Gaussian deformation residual refinement. Given the densified deformed set $\bar{\mathcal{G}}_t^+$, we use a deformation residual network f_{DR} to predict small parameter corrections for each Gaussian:

$$\left(\Delta\mathbf{p}_{n,t}^{\text{res}}, \Delta\mathbf{r}_{n,t}^{\text{res}}, \Delta\mathbf{s}_{n,t}^{\text{res}}\right) = f_{DR}\big(\gamma(\bar{\mathbf{p}}_{n,t}^+), \gamma(\bar{\mathbf{r}}_{n,t}^+), \gamma(\bar{\mathbf{s}}_{n,t}^+)\big), \tag{33}$$

where $(\bar{\mathbf{p}}_{n,t}^+, \bar{\mathbf{r}}_{n,t}^+, \bar{\mathbf{s}}_{n,t}^+)$ are the parameters of the n-th Gaussian in $\bar{\mathcal{G}}_t^+$, and $\gamma(\cdot)$ is applied component-wise.

The refined Gaussian parameters become

$$\tilde{\mathbf{p}}_{n,t} = \bar{\mathbf{p}}_{n,t}^+ + \Delta\mathbf{p}_{n,t}^{\text{res}}, \tag{34}$$

$$\tilde{\mathbf{r}}_{n,t} = \bar{\mathbf{r}}_{n,t}^+ + \Delta\mathbf{r}_{n,t}^{\text{res}}, \tag{35}$$

$$\tilde{\mathbf{s}}_{n,t} = \bar{\mathbf{s}}_{n,t}^+ + \Delta\mathbf{s}_{n,t}^{\text{res}}. \tag{36}$$

This yields the refined Gaussian set

$$\tilde{\mathcal{G}}_t = \{\tilde{g}_{n,t}\}_{n=1}^N, \tag{37}$$

which is splatted to obtain a base rendered image

$$\tilde{\mathbf{I}}_t = \mathcal{R}(\tilde{\mathcal{G}}_t). \tag{38}$$

During this stage, we freeze the base deformation module and optimize only f_{DR} with photometric supervision on the rendered image:

$$\mathcal{L}_{\text{base}} = (1-\lambda_{dssim})\mathcal{L}_1(\tilde{\mathbf{I}}_t, \mathbf{I}_t^{\text{gt}}) + \lambda_{dssim}\left(1 - \text{SSIM}(\tilde{\mathbf{I}}_t, \mathbf{I}_t^{\text{gt}})\right). \tag{39}$$

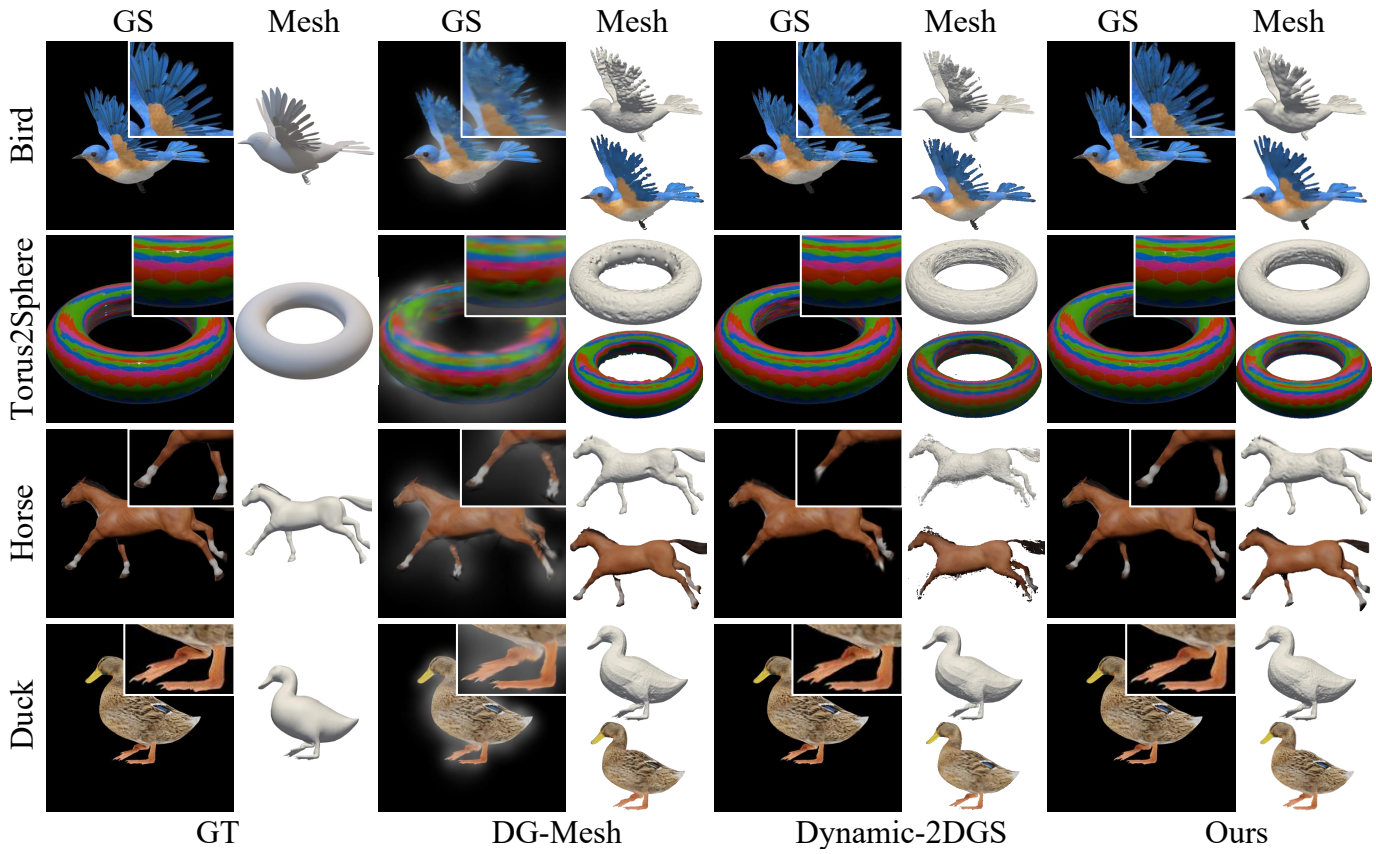


Fig. 5. **Qualitative results on DG-Mesh dataset.** For each method, we visualize both the direct GS rendering result and the final extracted mesh to evaluate performance on appearance and geometry. Our method demonstrates superior results in both categories, closely matching the ground-truth (GT).

Because the base deformation module is frozen, this stage is limited to local residual corrections around the geometry-stabilized base rather than re-optimizing the full 3D structure from scratch or relaxing the underlying mesh supervision.

Stage 2: Image residual refinement. Even after 3D residual refinement, some view-consistent high-frequency details remain difficult to encode through geometry-constrained 3D primitives alone. We therefore introduce an image-space residual branch to model only the remaining appearance discrepancy. In our implementation, this branch is instantiated with a Local Texture Estimator f_{lte} [45]. Once the 3D residual stage converges, we freeze f_{DR} and use the fixed base rendering $\tilde{\mathbf{I}}_t$ as the low-frequency image. Since Stage 2 operates at unit scale, the source and target grids coincide, and the skip connection reduces to the identity mapping on the fixed base rendering $\tilde{\mathbf{I}}_t$. Let $E(\cdot)$, $D(\cdot)$, and $h_{pha}(\cdot)$ denote the encoder, lightweight decoder, and phase estimator of the LTE-style branch, respectively, and let

$$\mathbf{Z}_t = E(\tilde{\mathbf{I}}_t) \tag{40}$$

be the latent feature map. Let $\mathbf{c}_t$ denote the fixed unit-scale cell corresponding to the resolution of $\tilde{\mathbf{I}}_t$, and define

$$\mathbf{v}_t^{pha} = h_{pha}(\mathbf{c}_t),$$

which is shared by all local neighbors at time t. For a query coordinate $\mathbf{x}$, let $\mathbf{x}_j$ denote the coordinate of the j-th local latent code in $\mathbf{Z}_t$, and define the relative coordinate as $\boldsymbol{\delta}_j = \mathbf{x} - \mathbf{x}_j$. We use the four nearest latent codes $\{\mathbf{z}_j\}_{j=1}^4$ with bilinear local-ensemble weights $w_j(\mathbf{x})$ satisfying $\sum_{j=1}^4 w_j(\mathbf{x}) = 1$. For each neighbor, the LTE branch predicts an amplitude vector $\mathbf{v}_j^{amp}$ and a frequency matrix $\mathbf{v}_j^{freq}$ from the local latent code $\mathbf{z}_j$, and maps the local coordinate into Fourier space as

$$H_\Psi(\mathbf{z}_j, \boldsymbol{\delta}_j) = \mathbf{v}_j^{amp} \odot \begin{bmatrix} \cos\big(\pi(\mathbf{v}_j^{freq}\boldsymbol{\delta}_j + \mathbf{v}_t^{pha})\big) \\ \sin\big(\pi(\mathbf{v}_j^{freq}\boldsymbol{\delta}_j + \mathbf{v}_t^{pha})\big) \end{bmatrix}, \tag{41}$$

where $\odot$ denotes element-wise multiplication. The decoder output from each neighboring code is blended by the local ensemble weights to produce a pixel residual:

$$\Delta\mathbf{I}_t(\mathbf{x}) = \sum_{j=1}^{4} w_j(\mathbf{x})\, D(H_\Psi(\mathbf{z}_j, \boldsymbol{\delta}_j)). \tag{42}$$

The final prediction retains the frozen 3D rendering as the low-frequency base and adds only the learned high-frequency correction:

$$\hat{\mathbf{I}}_t(\mathbf{x}) = \tilde{\mathbf{I}}_t(\mathbf{x}) + \Delta\mathbf{I}_t(\mathbf{x}). \tag{43}$$

The second stage trains only f_{lte} with

$$\begin{aligned} \mathcal{L}_{RGB} = (1-\lambda_{dssim})\mathcal{L}_1(\hat{\mathbf{I}}_t, \mathbf{I}_t^{\mathrm{gt}}) \\ + \lambda_{dssim}\left(1 - \mathrm{SSIM}(\hat{\mathbf{I}}_t, \mathbf{I}_t^{\mathrm{gt}})\right), \end{aligned} \tag{44}$$

where $\lambda_{dssim} = 0.2$.

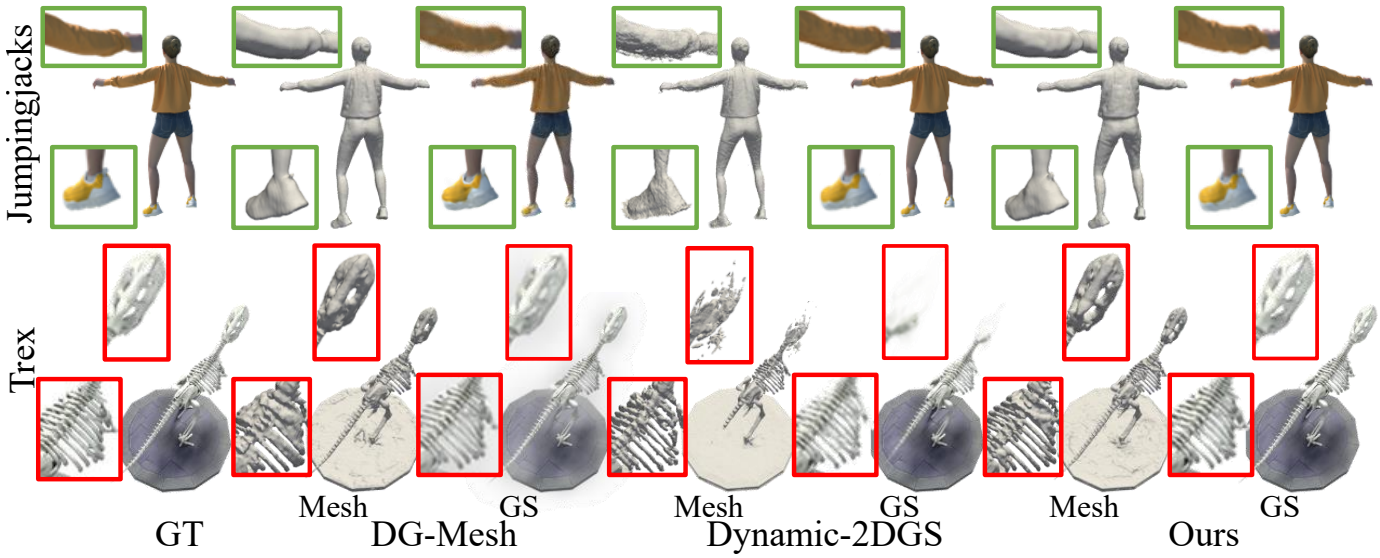


Fig. 6. **Qualitative results on D-NeRF dataset.**

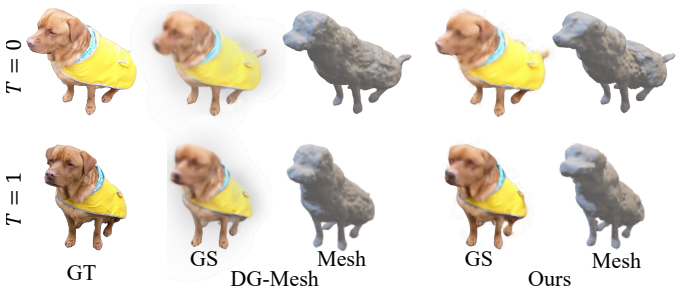


Fig. 7. **Qualitative results on Nerfies dataset.**

Optimization schedule. We first optimize the base representation with MCDM and SAAD until the geometry becomes stable under mesh supervision. RBER is then activated sequentially: we first train f_{DR} while freezing the base deformation model, and then freeze f_{DR} and train f_{lte}. This stage-wise schedule follows the structural reformulation introduced in Sec. I: temporal deformation is localized first, geometry-support bias is corrected second, and only the remaining appearance discrepancy is restored last. Consequently, the image residual branch is used to recover unresolved appearance details rather than compensate for incomplete 3D deformation or weak geometry.

V. Experiments

A. Experiment Settings

Datasets. We conduct our evaluations on three widely-used public datasets. First, we use the D-NeRF dataset [1], which consists of 8 synthetic monocular scenes featuring complex, non-rigid motions. This dataset is a standard benchmark for evaluating the quality of NVS in dynamic settings. Second, to quantitatively measure geometric accuracy, we use the DG-Mesh dataset [15], which provides ground-truth meshes for each frame across six different dynamic scenes. This allows for a direct and precise evaluation of mesh reconstruction quality. Third, we use the Nerfies dataset [2], which contains real-world monocular captures of deformable scenes with casual hand-held camera motion, to evaluate generalization to in-the-wild dynamic scenarios.

Implementation Details. Implemented in PyTorch on a single RTX 4090, we train for 50,000 iterations following standard Gaussian Splatting hyperparameters [18]. We begin with a global warm-up (1,000 iterations) to establish priors before cloning into K groups, where $K \in \{2, 3\}$ is based on the sequence length of each scene. Mesh extraction activates at 10,000 iterations, and the RBER begins at 25,000 iterations with the base deformation frozen. For mesh color rendering, we use Nvdiffrast [46].

B. Results Comparison

Quantitative Results. We evaluate our method on the DG-Mesh [15] and D-NeRF [1] datasets. Tab. I demonstrates that our framework establishes an optimal balance between visual and geometric fidelity on DG-Mesh. Ours matches or exceeds the geometry metrics(CD, EMD) of DG-Mesh while maintaining substantially higher PSNR, closely approaching Dynamic-2DGS [16] and achieving superior PSNR$_m$. Tab. II further shows that on D-NeRF, our approach achieves competitive NVS metrics (PSNR, SSIM, LPIPS) comparable to Dynamic-2DGS, while significantly surpassing all baselines in surface rendering quality, consistently yielding the highest PSNR$_m$ across the majority of scenes.

Qualitative Results. Fig. 5 shows that while DG-Mesh yields blurry renderings and Dynamic-2DGS produces fragmented surfaces (e.g., missing parts in *horse*), ours reconstructs sharp appearances with topologically clean meshes. On D-NeRF (Fig. 6), our method faithfully recovers intricate details like the thin *Trex* skeleton and clothing wrinkles in *Jumpingjacks*. Finally, on the real-world Nerfies dataset (Fig. 7), our approach consistently achieves sharper renderings and finer surface geometry than DG-Mesh under complex non-rigid motions.

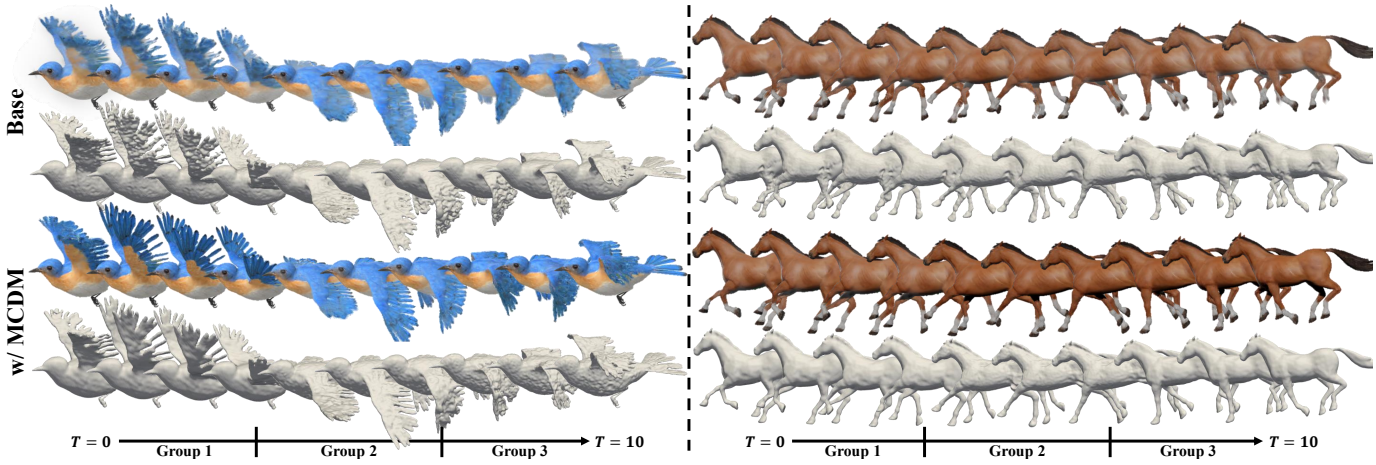


Fig. 8. **Effectiveness of MCDM.** The baseline (top two rows) and MCDM (bottom two rows) are compared across an extended temporal sequence ($T = 0, \ldots, 10$). Group boundaries are marked on the timeline.

TABLE III
QUANTITATIVE ABLATION STUDY. WE EVALUATE THE CONTRIBUTION OF EACH COMPONENT ON THE DG-MESH DATASET. METRICS INCLUDE PSNR, PSNR$_m$, CHAMFER DISTANCE (CD), AND EARTH MOVER'S DISTANCE (EMD).

Components			Metrics			
MCDM	SAAD	RBER	PSNR↑	PSNR$_m$↑	CD↓	EMD↓
			25.03	30.67	0.697	0.130
✓			29.69 (+4.66)	30.33 (-0.34)	0.695 (-0.002)	0.129 (-0.001)
✓	✓		34.07 (+9.04)	30.49 (-0.18)	0.673 (-0.024)	0.126 (-0.004)
✓	✓	✓	36.28 (+11.25)	32.88 (+2.21)	0.673 (-0.024)	0.126 (-0.004)

TABLE IV
MCDM GROUP-COUNT SWEEP (PLACEHOLDER). PSNR AND CD FOR $K \in \{1, 2, 3, 4, 5\}$ ON DG-MESH.

K	PSNR↑	CD↓
1	*TBD*	*TBD*
2	*TBD*	*TBD*
3	*TBD*	*TBD*
4	*TBD*	*TBD*
5	*TBD*	*TBD*

TABLE V
EFFECT OF GLOBAL WARM-UP LENGTH IN MCDM. WE VARY THE NUMBER OF WARM-UP ITERATIONS BEFORE CLONING THE GLOBAL CANONICAL SET INTO MULTIPLE CANONICAL GROUPS, AND REPORT RENDERING QUALITY AND MESH ACCURACY ON THE DG-MESH DATASET.

Warm-up Iter.	PSNR ↑	CD ↓
0k	XX.XX	0.XXXX
1k	XX.XX	0.XXXX
3k	XX.XX	0.XXXX
8k	XX.XX	0.XXXX
16k	XX.XX	0.XXXX

C. Ablation Study

We quantitatively validate the proposed staged design on the DG-Mesh dataset in Tab. III. MCDM primarily improves Gaussian rendering (+4.66 PSNR), indicating that localizing the temporal deformation burden reduces motion over-smoothing in a shared canonical space. Adding SAAD further improves the rendering–geometry balance, yielding cumulative changes of -0.024 CD, -0.004 EMD, and +9.04 PSNR over the baseline. Finally, RBER restores residual appearance details on top of the geometry-stabilized representation, bringing the total gains over the baseline to +11.25 PSNR and +2.21 PSNR$_m$ for Gaussian and mesh rendering, respectively. Taken together, these results support our staged design: MCDM localizes temporal deformation, SAAD corrects geometry-driven densification bias, and RBER recovers the remaining appearance discrepancy.

MCDM. The baseline forces a single canonical representation to explain the entire temporal deformation span. Because this shared canonical set must accommodate motions across all time steps, the deformation MLP is required to map one canonical space to diverse local motions, which leads to motion over-smoothing and blurred rendering (Fig. 8, top). MCDM redistributes this burden across multiple canonical Gaussian sets. Since each set is responsible only for a localized temporal segment, the deformation network operates over a reduced motion range and can model local dynamics more faithfully. As shown in Fig. 8 (bottom), this yields sharper appearances and more coherent reconstruction across the sequence. The gain from MCDM is therefore mainly attributable to better temporal localization rather than stronger geometry supervision.

SAAD. Under mesh supervision, missing surface regions must be filled by newly densified Gaussians. In the isotropic variant, these primitives are initialized at face centroids with isotropic scales that ignore the shape of the corresponding mesh faces (Fig. 10, top). As a result, densification increases geometric coverage, which can reduce CD, but the inserted support is poorly matched to local surface structure and progressively blurs the rendered result. This trend is visible in Fig. 9 (w/o, 10k→25k), where rendering quality degrades as isotropically initialized Gaussians accumulate. SAAD instead derives the initial covariance of each new primitive from local face statistics, aligning its support with the tangent structure of the surface from initialization. This turns densification from a purely coverage-seeking operation into a surface-aligned

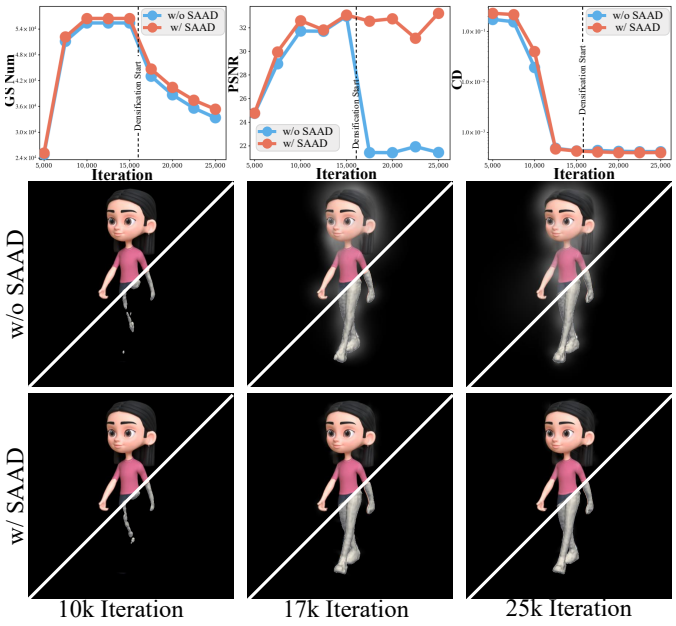


Fig. 9. **Training progress of SAAD.** (Top) Gaussian count, PSNR, and CD curves over training iterations for the baseline (blue) and SAAD (red). The dashed line marks the start of densification. (Bottom) Visual comparison at 10k, 17k, and 25k iterations with and without SAAD.

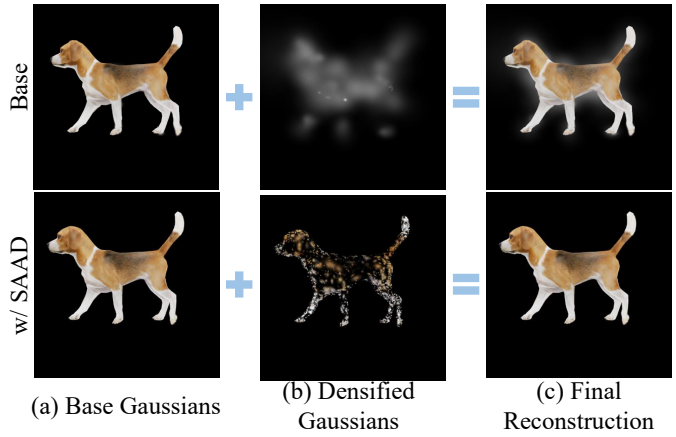


Fig. 10. **Visual comparison of SAAD.** Baseline (top) produces diffuse primitives (b), blurring the reconstruction (c). SAAD (bottom) generates surface-aligned primitives (b), yielding sharp boundaries (c).

support refinement process. Consequently, SAAD improves geometry while better preserving rendering quality, and produces sharper boundaries and cleaner local structure in Fig. 10 (bottom).

RBER. After MCDM and SAAD, the representation becomes geometry-dominant: Gaussian centers and orientations are strongly tied to the extracted surface, which leaves limited residual freedom for fine appearance details. RBER restores this missing appearance capacity without re-optimizing the underlying geometry. As shown in Fig. 11, the 3D residual branch f_{DR} introduces only local parameter corrections on top of the frozen geometry-stabilized base, reducing the L1 error from (a) to (b). The subsequent image-space residual branch f_{lte} then models the remaining high-frequency discrepancy, recovering fine textures such as wing feathers from (b) to

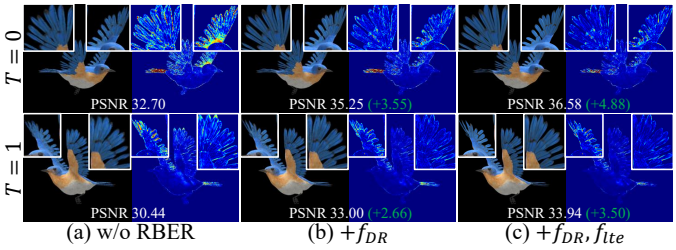


Fig. 11. **Ablation of RBER.** Each column shows the rendered image (left) and its L1 error map against the ground-truth (right) at two time steps ($T = 0$, $T = 1$). From (a) to (c), each module is progressively added, with zoom-in patches highlighting the error reduction in fine-detail regions.

(c). The consistent PSNR gains at both time steps ($T = 0$: +4.88, $T = 1$: +3.50) indicate that RBER improves fine-scale appearance while retaining the geometric consistency established by the preceding stages.

VI. Conclusions

We have presented a unified framework that mitigates the trade-off between rendering fidelity and geometric accuracy in dynamic 3D Gaussian reconstruction. MCDM localizes motion modeling via multiple canonical Gaussian sets. SAAD replaces isotropic densification with surface-aligned, geometry-aware initialization derived from local face statistics. RBER restores fine-scale appearance through sequential 3D and 2D residual refinement on top of a geometry-stabilized base. Experiments on DG-Mesh, D-NeRF, and Nerfies show improved rendering quality together with preserved or improved mesh accuracy, indicating a better rendering–geometry balance across diverse dynamic scenes.

References

[1] A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer, "D-nerf: Neural radiance fields for dynamic scenes," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 10 318–10 327.

[2] K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R. Martin-Brualla, "Nerfies: Deformable neural radiance fields," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 5865–5874.

[3] S. Fridovich-Keil, G. Meanti, F. R. Warburg, B. Recht, and A. Kanazawa, "K-planes: Explicit radiance fields in space, time, and appearance," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 12 479–12 488.

[4] A. Cao and J. Johnson, "Hexplane: A fast representation for dynamic scenes," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 130–141.

[5] J. Fang, T. Yi, X. Wang, L. Xie, X. Zhang, W. Liu, M. Nießner, and Q. Tian, "Fast dynamic radiance fields with time-aware neural voxels," in *SIGGRAPH Asia 2022 Conference Papers*, 2022, pp. 1–9.

[6] Y. Du, Y. Zhang, H.-X. Yu, J. B. Tenenbaum, and J. Wu, "Neural radiance flow for 4d view synthesis and video processing," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE Computer Society, 2021, pp. 14 304–14 314.

[7] C. Gao, A. Saraf, J. Kopf, and J.-B. Huang, "Dynamic view synthesis from dynamic monocular video," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 5712–5721.

[8] Z. Li, S. Niklaus, N. Snavely, and O. Wang, "Neural scene flow fields for space-time view synthesis of dynamic scenes," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 6498–6508.

[9] W. Xian, J.-B. Huang, J. Kopf, and C. Kim, "Space-time neural irradiance fields for free-viewpoint video," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021, pp. 9421–9431.

[10] Z. Yang, X. Gao, W. Zhou, S. Jiao, Y. Zhang, and X. Jin, "Deformable 3d gaussians for high-fidelity monocular dynamic scene reconstruction," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 20 331–20 341.

[11] G. Wu, T. Yi, J. Fang, L. Xie, X. Zhang, W. Wei, W. Liu, Q. Tian, and X. Wang, "4d gaussian splatting for real-time dynamic scene rendering," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 20 310–20 320.

[12] J. Luiten, G. Kopanas, B. Leibe, and D. Ramanan, "Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis," in *2024 International Conference on 3D Vision (3DV)*. IEEE, 2024, pp. 800–809.

[13] Z. Li, Z. Chen, Z. Li, and Y. Xu, "Spacetime gaussian feature splatting for real-time dynamic view synthesis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 8508–8520.

[14] Z. Yang, H. Yang, Z. Pan, and L. Zhang, "Real-time photorealistic dynamic scene representation and rendering with 4d gaussian splatting," *arXiv preprint arXiv:2310.10642*, 2023.

[15] I. Liu, H. Su, and X. Wang, "Dynamic gaussians mesh: Consistent mesh reconstruction from dynamic scenes," *arXiv preprint arXiv:2404.12379*, 2024.

[16] S. Zhang, G. Wu, Z. Xie, X. Wang, B. Feng, and W. Liu, "Dynamic 2d gaussians: Geometrically accurate radiance fields for dynamic objects," *arXiv preprint arXiv:2409.14072*, 2024.

[17] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.

[18] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering." *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.

[19] K. Park, U. Sinha, P. Hedman, J. T. Barron, S. Bouaziz, D. B. Goldman, R. Martin-Brualla, and S. M. Seitz, "Hypernerf: A higher-dimensional representation for topologically varying neural radiance fields," *arXiv preprint arXiv:2106.13228*, 2021.

[20] E. Tretschk, A. Tewari, V. Golyanik, M. Zollhöfer, C. Lassner, and C. Theobalt, "Non-rigid neural radiance fields: Reconstruction and novel view synthesis of a dynamic scene from monocular video," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 12 959–12 970.

[21] R. Shao, Z. Zheng, H. Tu, B. Liu, H. Zhang, and Y. Liu, "Tensor4d: Efficient neural 4d decomposition for high-fidelity dynamic reconstruction and rendering," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 16 632–16 642.

[22] Z. Xu, S. Peng, H. Lin, G. He, J. Sun, Y. Shen, H. Bao, and X. Zhou, "4k4d: Real-time 4d view synthesis at 4k resolution," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 20 029–20 040.

[23] Y.-H. Huang, Y.-T. Sun, Z. Yang, X. Lyu, Y.-P. Cao, and X. Qi, "Sc-gs: Sparse-controlled gaussian splatting for editable dynamic scenes," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 4220–4230.

[24] Y. Xue, B. L. Bhatnagar, R. Marin, N. Sarafianos, Y. Xu, G. Pons-Moll, and T. Tung, "Nsf: Neural surface fields for human modeling from monocular depth," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2023, pp. 15 049–15 060.

[25] R. Hanocka, G. Metzer, R. Giryes, and D. Cohen-Or, "Point2mesh: A self-prior for deformable meshes," *arXiv preprint arXiv:2005.11084*, 2020.

[26] J. Kaltheuner, H. Dröge, M. Plack, P. Stotko, and R. Klein, "Neu-pig: Neural preconditioned grids for fast dynamic surface reconstruction on long sequences," *arXiv preprint arXiv:2602.22212*, 2026.

[27] J. Pan, X. Han, W. Chen, J. Tang, and K. Jia, "Deep mesh reconstruction from single rgb images via topology modification networks," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 9964–9973.

[28] A. Kanazawa, S. Tulsiani, A. A. Efros, and J. Malik, "Learning category-specific mesh reconstruction from image collections," in *Proceedings of the European conference on computer vision (ECCV)*, 2018, pp. 371–386.

[29] X. Li, S. Liu, S. De Mello, K. Kim, X. Wang, M.-H. Yang, and J. Kautz, "Online adaptation for consistent mesh reconstruction in the wild," *Advances in Neural Information Processing Systems*, vol. 33, pp. 15 009–15 019, 2020.

[30] G. Yang, M. Vo, N. Neverova, D. Ramanan, A. Vedaldi, and H. Joo, "Banmo: Building animatable 3d neural models from many casual videos," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 2863–2873.

[31] G. Yang, S. Yang, J. Z. Zhang, Z. Manchester, and D. Ramanan, "Ppr: Physically plausible reconstruction from monocular videos," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 3914–3924.

[32] W. Mao, R. Hartley, M. Salzmann *et al.*, "Neural sdf flow for 3d reconstruction of dynamic scenes," in *The Twelfth International Conference on Learning Representations*, 2024.

[33] H. Cai, W. Feng, X. Feng, Y. Wang, and J. Zhang, "Neural surface reconstruction of dynamic scenes with monocular rgb-d camera," *Advances in Neural Information Processing Systems*, vol. 35, pp. 967–981, 2022.

[34] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3d surface construction algorithm," in *Seminal graphics: pioneering efforts that shaped the field*, 1998, pp. 347–353.

[35] A. Guédon and V. Lepetit, "Sugar: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 5354–5363.

[36] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2d gaussian splatting for geometrically accurate radiance fields," in *ACM SIGGRAPH 2024 conference papers*, 2024, pp. 1–11.

[37] Q. Zhou, Y. Gong, W. Yang, J. Li, Y. Luo, B. Xu, S. Li, B. Fei, and Y. He, "Mgsr: 2d/3d mutual-boosted gaussian splatting for high-fidelity surface reconstruction under various light conditions," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 27 295–27 304.

[38] Z. Yu, T. Sattler, and A. Geiger, "Gaussian opacity fields: Efficient adaptive surface reconstruction in unbounded scenes," *ACM Transactions on Graphics (ToG)*, vol. 43, no. 6, pp. 1–13, 2024.

[39] D. Chen, H. Li, W. Ye, Y. Wang, W. Xie, S. Zhai, N. Wang, H. Liu, H. Bao, and G. Zhang, "Pgsr: Planar-based gaussian splatting for efficient and high-fidelity surface reconstruction," *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 9, pp. 6100–6111, 2024.

[40] Z. Gao, J.-W. Bian, G. Lin, H. Chen, and C. Shen, "Surfacesplat: Connecting surface reconstruction and gaussian splatting," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2025, pp. 28 525–28 534.

[41] S. Ma, Y. Luo, W. Yang, and Y. Yang, "Mags: Reconstructing and simulating dynamic 3d objects with mesh-adsorbed gaussian splatting," *arXiv preprint arXiv:2406.01593*, 2024.

[42] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, S. Hodges, and A. Fitzgibbon, "Kinectfusion: Real-time dense surface mapping and tracking," in *2011 10th IEEE international symposium on mixed and augmented reality*. Ieee, 2011, pp. 127–136.

[43] S. Peng, C. Jiang, Y. Liao, M. Niemeyer, M. Pollefeys, and A. Geiger, "Shape as points: A differentiable poisson solver," *Advances in Neural Information Processing Systems*, vol. 34, pp. 13 032–13 044, 2021.

[44] X. Wei, F. Xiang, S. Bi, A. Chen, K. Sunkavalli, Z. Xu, and H. Su, "Neumanifold: Neural watertight manifold reconstruction with efficient and high-quality rendering support," in *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2025, pp. 731–741.

[45] J. Lee and K. H. Jin, "Local texture estimator for implicit representation function," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 1929–1938.

[46] S. Laine, J. Hellsten, T. Karras, Y. Seol, J. Lehtinen, and T. Aila, "Modular primitives for high-performance differentiable rendering," *ACM Transactions on Graphics*, vol. 39, no. 6, 2020.